SQL - 04 - Group By

Problem Statement:

You are a Data Analyst at **Food Corporation of India** (FCI). You have been tasked to study the Farmer's market - *Mandis*.

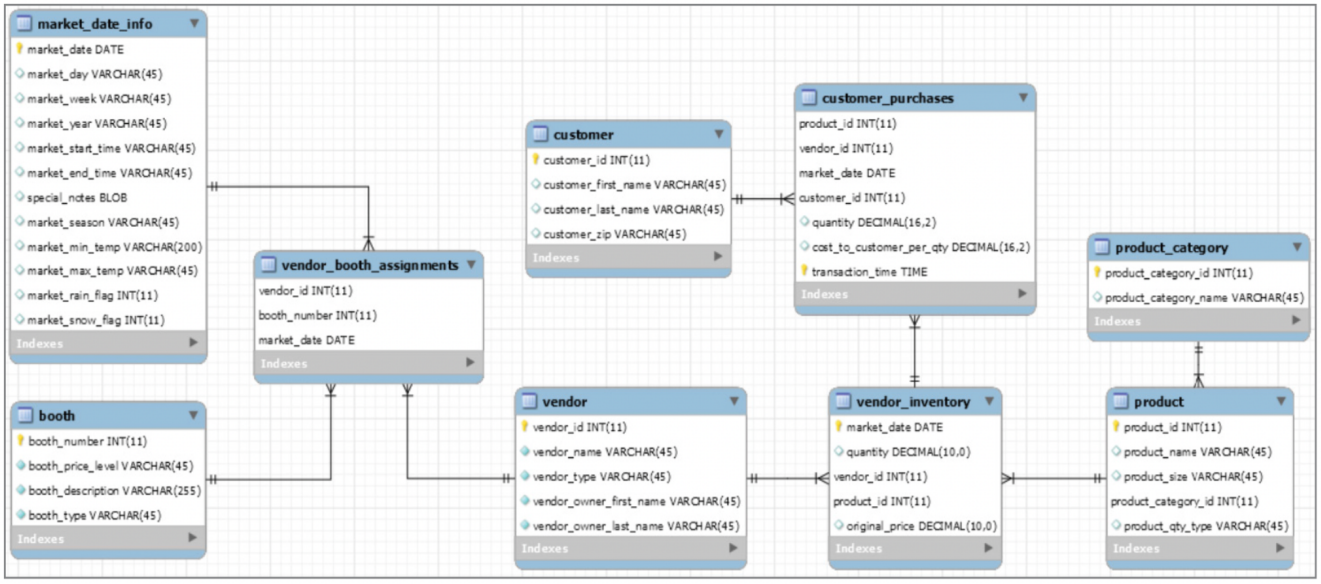
Now, in a farmer's market, you'll have multiple things:

1. Customers
2. Vendors
3. Booths/Stalls
4. Products <-> Product Categories
5. Market information - date, day, week, start time, end time, rain, etc.

So, the story here will become:

- The market opens up.
- Vendors get their slots assigned to them.
- Vendors will also maintain their inventory
- In the inventory, vendors will have multiple products to sell each of which will belong to a category, stored in the product category

Dataset: Farmer's Market Database



SQL - 04 - Aggregating Results - Group BY

SQL starts becoming powerful for analysis when you use it to aggregate data.

Using the GROUP BY statement, you can specify the level of summarization and then use aggregate functions to summarize values for the records in each group.

- Data analysts can use SQL to build dynamic summary reports that can be automatically updated.
- Dashboards and reports built using software like Tableau etc often rely on SQL queries to get the data they need from the underlying database in an aggregated form that can be used for reporting.

But it all starts with primary SQL aggregation.

GROUP BY Syntax

```
SELECT [columns to return]
FROM [table]
WHERE [conditional filter statements]
GROUP BY [columns to group on]
HAVING [conditional filter statements that are run after grouping]
ORDER BY [columns to sort on]
```

The GROUP BY keywords are followed by a comma-separated list of column names that indicate how you want to summarize the query results.

Important Points:

- GROUP BY clause is used with the SELECT statement.
- In the query, GROUP BY clause is placed after the WHERE clause.

- In the query, GROUP BY clause is placed before ORDER BY clause if used any.

Question: Get a list of the customer IDs who made purchases on each market date.

If I write this…

```
SELECT
    market_date,
    customer_id
 FROM farmers_market.customer_purchases
 ORDER BY market_date, customer_id
```

- One row per item each customer purchased, **displaying duplicates in the output**, because you're querying the customer_purchases table with no grouping specified.

```
SELECT
    market_date,
    customer_id
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
 ORDER BY market_date, customer_id
```

- You can also accomplish the same result by using SELECT DISTINCT to remove duplicates,
- We are using GROUP BY to add summary columns to the output.

Aggregate functions - SUM() and COUNT()

Question: Count the number of purchases each customer made per market date.

```
SELECT
    market_date,
    customer_id,
    COUNT(*) AS num_purchases
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
 ORDER BY market_date, customer_id
 LIMIT 10
```

Keep an eye on **granularity** of the table data

- The granularity of the **customer_purchases** table is such that if a customer were to buy three identical items, such as tomatoes, at once from a vendor, that would show up as 1 in the ***num_purchases*** column of this query's output, since the item purchase is recorded in one row in the table, with a quantity value of 3.
- If the customer were to buy three tomatoes, walk away from the stand, then go back and purchase another three tomatoes, that would be counted as two by the preceding query, since the new separate purchase would generate a new line in the database.

Alternate Question: Calculate the total quantity purchased by each customer per market_date.

Here, we'll use the SUM() function

```
SELECT
    market_date,
    customer_id,
    SUM(quantity) AS total_qty_purchased
```

```
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
 ORDER BY market_date, customer_id
 LIMIT 10
```

- **Note** : It's important to **understand the granularity and structure** of the underlying table to ensure that your result means what you think it does.

- Thus, always write the query without aggregation first to see the values you will be summarizing before grouping the results.

Slightly complex question: how many different kinds of products were purchased by each customer on each market date:

Different kinds of products

```
SELECT
    market_date,
    customer_id,
    COUNT(DISTINCT product_id) AS different_products_purchased
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
LIMIT 10
```

- So instead of counting how many rows there were in the customer_purchases table per customer per market date, like we did with COUNT(*),
- or adding up the quantities, like we did with SUM(quantity),
- we're identifying **how many unique product_id** values exist across those rows in the group

Add the qty summary to the same query:

```
SELECT
```

```
    market_date,
    customer_id,
    SUM(quantity) AS total_qty_purchased,
    COUNT(DISTINCT product_id) AS different_products_purchased
FROM farmers_market.customer_purchases
GROUP BY market_date, customer_id
ORDER BY market_date, customer_id
```

Question: Calculate the total price paid by customer_id 3 per market_date.

Can perform Calculations Inside Aggregation Functions

```
SELECT
        customer_id,
        market_date,
        SUM(quantity * cost_to_customer_per_qty) AS total_spent
    FROM farmers_market.customer_purchases
    WHERE
        customer_id = 3
    GROUP BY market_date
    ORDER BY market_date
```

- The price will be calculated per row of the table, and then the results will be summed up per group.

- **Important :** Notice that **vendor_id has been removed from the list of columns** to be displayed and from the ORDER BY clause.
- That's because if we want the aggregation level of one row per customer per date, we **can't also include vendor_id** in the output, **because the customer can purchase from multiple vendors on a single date**, so the results wouldn't be aggregated at the level we wanted.

Question: What if we wanted to find out how much this customer had spent at each vendor, regardless of date?

Then we can group by customer_id and vendor_id:

```
SELECT
    customer_id,
    vendor_id,
    SUM(quantity * cost_to_customer_per_qty) AS total_spent
 FROM farmers_market.customer_purchases
 GROUP BY customer_id, vendor_id
 ORDER BY customer_id, vendor_id
```

Let's add some customer details and vendor details to these results. Customer details are in the customer table and vendor details are in the vendor table. - So that means we will have to JOIN

```
 SELECT
    c.customer_first_name,
    c.customer_last_name,
    cp.customer_id,
    v.vendor_name,
    cp.vendor_id,
    cp.quantity * cp.cost_to_customer_per_qty AS price
 FROM farmers_market.customer c
    LEFT JOIN farmers_market.customer_purchases cp
        ON c.customer_id = cp.customer_id
    LEFT JOIN farmers_market.vendor v
        ON cp.vendor_id = v.vendor_id
 ORDER BY cp.customer_id, cp.vendor_id
```

- To summarize at the level of one row per customer per vendor, we will have to group by a lot more fields, including all of the customer table fields and all of the vendor table fields.

- Basically, we want to group by all of the displayed fields that don't include aggregate functions.

```
SELECT
    c.customer_first_name,
    c.customer_last_name,
    cp.customer_id,
    v.vendor_name,
    cp.vendor_id,
    ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS total_spent
 FROM farmers_market.customer c
    LEFT JOIN farmers_market.customer_purchases cp
        ON c.customer_id = cp.customer_id
    LEFT JOIN farmers_market.vendor v
        ON cp.vendor_id = v.vendor_id
 WHERE
    cp.customer_id = 3
 GROUP BY
    cp.customer_id,
    cp.vendor_id
 ORDER BY cp.customer_id, cp.vendor_id
```

Question: We want to get the most and least expensive items per product category, considering the fact that each vendor sets their own prices and can adjust prices per customer.

We will use the **vendor_inventory** table, which has a field for the original price the vendors set for each item they bring to market on each market date.

First, let's look at all of the available fields in the vendor_inventory table by using select * query.

```
SELECT *
  FROM farmers_market.vendor_inventory
  ORDER BY original_price
  LIMIT 10
```

MIN and MAX

We can get the least and most expensive item prices in the entire table by using the MIN() and MAX() functions without grouping in MySQL

```
SELECT
    MIN(original_price) AS minimum_price,
    MAX(original_price) AS maximum_price
  FROM farmers_market.vendor_inventory
  ORDER BY original_price
```

- But if we want to **get the lowest and highest prices within each product category**, we have to group by the product_category_id,
- then the summary values will be calculated per group,

```
SELECT
    pc.product_category_name,
    p.product_category_id,
    MIN(vi.original_price) AS minimum_price,
    MAX(vi.original_price) AS maximum_price
  FROM farmers_market.vendor_inventory AS vi
    INNER JOIN farmers_market.product AS p
        ON vi.product_id = p.product_id
    INNER JOIN farmers_market.product_category AS pc
        ON p.product_category_id = pc.product_category_id
  GROUP BY pc.product_category_name, p.product_category_id
```

- Get prices from vendor_inventory
- Products from the product table

- Their Categories from the product_category table.

Question: Count how many products were for sale on each market date, or how many different products each vendor offered.

We can determine these values using COUNT and COUNT DISTINCT.

COUNT will count up the rows within a group when used with GROUP BY, and COUNT DISTINCT will count up the unique values present in the specified field within the group.

- To determine how many products are offered for sale each market date, we can count up the rows in the **vendor_inventory** table, grouped by date.
- This doesn't tell us what quantity of each product was offered or sold, but counts the number of products available, because there is a row in this table for each product for each vendor for each market date.

```
SELECT
    market_date,
    COUNT(product_id) AS product_count
 FROM farmers_market.vendor_inventory
 GROUP BY market_date
 ORDER BY market_date
```

If we wanted to know how many different products—with unique product IDs—each vendor brought to market during a date range, we could use COUNT DISTINCT on the product_id field, like so:

```
SELECT
    vendor_id,
    COUNT(DISTINCT product_id) AS different_products_offered
```

```
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id
ORDER BY vendor_id
```

Question: In addition to the count of different products per vendor, we also want the **average original price of a product per vendor**?

We can add a line to the preceding query, and use the AVG() function

```
SELECT
    vendor_id,
    COUNT( product_id) AS different_products_offered,
    AVG(original_price) AS average_product_price
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id, product_id
 ORDER BY vendor_id
```

But is this average product price??

- Is it fair to call it “average product price” when the underlying table has one row per type of product?
- If the vendor brought 100 tomatoes to market, those would all be in one line of the underlying vendor inventory table, so the price of a tomato would only be included in the average once.
- If you calculated the “average product price” for the vendor this way, you would just get the average of the price of one tomato and one bouquet.

How to calculate the price per item?

- To get an **actual average price of items in each vendor’s inventory** between the specified dates, it might make more sense to **multiply the quantity** of each type of item times the price of that item, which is a

calculation that would occur per row, then sum that up and divide by the total quantity of items, which is a calculation that would occur per vendor.

- Let's try a calculation that includes these two summary values.

```
SELECT
    vendor_id,
    COUNT(DISTINCT product_id) AS different_products_offered,
    SUM(quantity * original_price) AS value_of_inventory,
    SUM(quantity) AS inventory_item_count,
    ROUND(SUM(quantity * original_price) / SUM(quantity), 2) AS
 average_item_price
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id
 ORDER BY vendor_id
```

Filtering with Having

Filtering is another thing that can be done in the query after summarization occurs.

If you want to filter values after the aggregate functions are applied, you can add a HAVING clause to the query. This filters the groups based on the summary values.

Question: filter out vendors who brought at least 10 items to the farmer's market over the time period - 2019-05-02 and 2019-05-16

```
SELECT
    vendor_id,
    COUNT(DISTINCT product_id) AS different_products_offered,
    SUM(quantity * original_price) AS value_of_inventory,
```

```
    SUM(quantity) AS inventory_item_count,
    SUM(quantity * original_price) / SUM(quantity) AS average_item_price
  FROM farmers_market.vendor_inventory
  WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
  GROUP BY vendor_id
  HAVING inventory_item_count >= 100
  ORDER BY vendor_id
```

GROUP BY, HAVING, and WHERE

Question: Find the average amount spent by customers on each day. We want to consider only those days where the number of purchases were more than 3 and the transaction amount was greater than 5.

```
SELECT
  market_date,
  ROUND(AVG(quantity * cost_to_customer_per_qty), 2) AS avg_spent
FROM customer_purchases
WHERE quantity * cost_to_customer_per_qty > 5
GROUP BY market_date
HAVING COUNT(*) > 3
ORDER BY market_date;
```